DS HW 4

1. Implement a Stack using Queue data structure:

```
#include <stdio.h>
#define MAX 100
int queue[MAX]; int front = 0, rear = -1;
void enqueue(int x) {
        queue[++rear] = x;
}
int dequeue() {
        return queue[front++];
}
 void push(int x) {
         int i, n, temp;
        n = rear - front + 1;

        enqueue(x);

        for (i = 0; i < n; i++)
        {
                temp = dequeue();
                enqueue(temp);
        }
}
int pop() {
        if (front > rear){
                 printf("Stack Underflow\n");
                  return -1;
        }
```

```
        return dequeue();
}
void display() {
int i;
if (front > rear)
{
   printf("Stack is empty\n");
   return;
}

for (i = front; i <= rear; i++)
   printf("%d ", queue[i]);

printf("\n");
}
int main() {
 push(10);
push(20);
push(30);
printf("Stack: ");
display();

printf("Popped element: %d\n", pop());

printf("Stack after pop: ");
display();

return 0;
}
```

OUTPUT:

Stack: 30 20 10

Popped element: 30

Stack after pop: 20 10

2. Implement a Queue using Stack data structure:

```
#include<stdio.h>
#define N 50
int s1[N],s2[N],top1=-1,top2=-1;
void push1(int x){
if(top1==N-1)
      printf("\nOverflow");
else
   s1[++top1]=x;
}
void push2(int x){
if(top2==N-1)
      printf("\nOverflow");
else
   s2[++top2]=x;
}
int pop1()
{
return s1[top1--];
}
int pop2()
{
return s2[top2--];
```

```
}
void enqueue(int x){
push1(x);
}
void dequeue(){
int a,b;
if(top1==-1 && top2==-1)
   printf("\nUnderflow");
else{
   while(top1!=-1){
      a=pop1();
      push2(a);

   }
   b=pop2();
   printf("\nDequeued element:%d",b);
   while(top2!=-1){
      a=pop2();
      push1(a);
   }

}
```

```
}
void display(){
printf("\nQueue :");
for(int i=0;i<=top1;i++){
    printf("%d\t",s1[i]);
}
}
int main(){
enqueue(10);
enqueue(20);
enqueue(30);
display();
 dequeue();
display();
return 0;
}
```

OUTPUT:

Queue :102030

Dequeued element:10

Queue :2030

3. Browser History using Stack:

```
#include <stdio.h>
#include <string.h>
```

```
#define MAX 100
char history[MAX][100];
int top = -1;

void visit(char page[])
{
   strcpy(history[++top], page);
}

void back()
{
   if (top <= 0)
   {
      printf("No previous page\n");
      return;
   }

   top--;
   printf("Back to: %s\n", history[top]);
}

void currentPage()
{
   if (top == -1)
      printf("No page visited\n");
   else
      printf("Current page: %s\n", history[top]);
```

```
}

int main()
{
    visit("Google");
    visit("YouTube");
    visit("Wikipedia");

    currentPage();

    back();
    currentPage();

    back();
    currentPage();

    return 0;
}
```

OUTPUT:

```
Current page: Wikipedia
Back to: YouTube
Current page: YouTube
Back to: Google
Current page: Google
```

4. Sort a Stack using Temporary Stack:

```
#include <stdio.h>
```

```
#define MAX 100

int s1[MAX], s2[MAX];
int top1 = -1, top2 = -1;

void push1(int x)
{
   s1[++top1] = x;
}
void push2(int x)
{
   s2[++top2] = x;
}

int pop1()
{
   return s1[top1--];
}

int pop2()
{
   return s2[top2--];
}

void sortStack()
{
```

```
    int x,a;

    while (top1 != -1)
    {
        x = pop1();

        while (top2 != -1 && s2[top2] > x)
        {
            a=pop2() ;
            push1(a);

        }

        push2(x);
    }

    while (top2 != -1)
    {
        a=pop2();
        push1(a);

    }
}

void display()
{
    int i;
```

```
    for (i = top1; i >= 0; i--)
        printf("%d ", s1[i]);

    printf("\n");
}

int main()
{
    push1(34);
    push1(3);
    push1(31);
    push1(98);
    push1(92);
    push1(23);

    printf("Original stack: ");
    display();

    sortStack();

    printf("Sorted stack: ");
    display();

    return 0;
}
```

OUTPUT:

Original stack: 23 92 98 31 3 34

Sorted stack: 3 23 31 34 92 98

5. Stock Span Problem:

```
#include <stdio.h>
int main()
{
    int price[] = {100, 80, 60, 70, 60, 75, 85};
    int span[7];
    int stack[7];
    int top = -1;
    int i, j;

    for (i = 0; i < 7; i++)
    {
        while (top >= 0 && price[stack[top]] <= price[i])
        {
            top--;
        }

        if (top == -1)
        {
            span[i] = i + 1;
        }
        else
        {
            span[i] = i - stack[top];
```

```
        }

        stack[++top] = i;
    }

    printf("Stock Span: ");

    for (i = 0; i < 7; i++)
    {
        printf("%d ", span[i]);
    }

    return 0;
}
```

OUTPUT:

Stock Span: 1 1 1 2 1 4 6

6. Nearest element to right having higher frequency:

```
#include <stdio.h>

#define MAX 100

int main()
{
    int a[] = {2, 1, 1, 3, 2, 1, 3};
    int n = 7;
```

```
int freq[10] = {0};
int stack[MAX];
int top = -1;
int result[MAX];
int i;

// Find frequency of each element
for (i = 0; i < n; i++)
    freq[a[i]]++;

// Find nearest element to right with higher frequency
for (i = n - 1; i >= 0; i--)
{
    while (top != -1 && freq[a[stack[top]]] <= freq[a[i]])
        top--;

    if (top == -1)
        result[i] = -1;
    else
        result[i] = a[stack[top]];

    stack[++top] = i;
}

printf("Result: ");

for (i = 0; i < n; i++)
```

```
        printf("%d ", result[i]);

    return 0;
}
```

OUTPUT:

Result: 1 -1 -1 1 1 -1 –1

7. Pattern containing I and D:

```
#include <stdio.h>
#include <string.h>

#define MAX 100

int stack[MAX];
int top = -1;

void push(int x)
{
    stack[++top] = x;
}

int pop()
{
    return stack[top--];
}
```

```
int main()
{
    char pattern[] = "DIDI";
    int n = strlen(pattern);
    int i, num = 1;

    for (i = 0; i <= n; i++)
    {
        push(num++);

        if (i == n || pattern[i] == 'I')
        {
            while (top != -1)
                printf("%d", pop());
        }
    }

    return 0;
}
```

OUTPUT:

21435

8. Minimum possible sum of two numbers:

```
#include <stdio.h>

void sort(int a[], int n)
{
    int i, j, temp;
```

```
    for (i = 0; i < n - 1; i++)
    {
        for (j = i + 1; j < n; j++)
        {
            if (a[i] > a[j])
            {
                temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
        }
    }
}

int main()
{
    int a[] = {6, 8, 4, 5, 2, 3};
    int n = 6;
    int num1 = 0, num2 = 0;
    int i;

    sort(a, n);

    for (i = 0; i < n; i++)
    {
        if (i % 2 == 0)
```

```
            num1 = num1 * 10 + a[i];
        else
            num2 = num2 * 10 + a[i];
    }

    printf("Numbers: %d and %d\n", num1, num2);
    printf("Minimum sum: %d\n", num1 + num2);

    return 0;
}
```

OUTPUT:

Numbers: 246 and 358

Minimum sum: 604

9. First negative integer in every window of size K:

```
#include <stdio.h>
#define MAX 100
int main()
{
    int a[] = {-8, 2, 3, -6, 10};
    int n = 5;
    int k = 2;

    int queue[MAX];
    int front = 0, rear = -1;
    int i;
```

```
    for (i = 0; i < n; i++)
    {

        if (a[i] < 0)
            queue[++rear] = i;

        if (front <= rear && queue[front] <= i - k)
            front++;

        // Print first negative number
        if (i >= k - 1)
        {
            if (front <= rear)
                printf("%d ", a[queue[front]]);
            else
                printf("0 ");
        }
    }

    return 0;
}
```

OUTPUT:

-8 0 -6 -6

10. Rotten Oranges using Queue:

```
#include <stdio.h>
```

```
#define MAX 100
struct Cell
{
   int row;
   int col;
};

int main()
{
   int a[3][5] =
   {
      {2, 1, 0, 2, 1},
      {1, 0, 1, 2, 1},
      {1, 0, 0, 2, 1}
   };

   struct Cell queue[MAX];
   int front = 0, rear = 0;

   int rows = 3, cols = 5;
   int fresh = 0;
   int time = 0;
   int i, j;

   int dr[] = {-1, 1, 0, 0};
   int dc[] = {0, 0, -1, 1};
```

```
for (i = 0; i < rows; i++)
{
   for (j = 0; j < cols; j++)
   {
      if (a[i][j] == 2)
      {
         queue[rear].row = i;
         queue[rear].col = j;
         rear++;
      }
      else if (a[i][j] == 1)
      {
         fresh++;
      }
   }
}

while (front < rear && fresh > 0)
{
   int size = rear - front;

   for (i = 0; i < size; i++)
   {
      int r = queue[front].row;
      int c = queue[front].col;
      front++;
```

```
        for (j = 0; j < 4; j++)
        {
            int nr = r + dr[j];
            int nc = c + dc[j];

            if (nr >= 0 && nr < rows &&
                nc >= 0 && nc < cols &&
                a[nr][nc] == 1)
            {
                a[nr][nc] = 2;
                fresh--;

                queue[rear].row = nr;
                queue[rear].col = nc;
                rear++;
            }
        }
    }

    time++;
}

if (fresh == 0)
    printf("Minimum time: %d\n", time);
else
    printf("All oranges cannot be rotten\n");
```

```
    return 0;
}
```

OUTPUT:

```
Minimum time: 2
```